

```

'''
Here is the complete, functional Exam Management System using **Tkinter** and **MySQL**.
The code is written procedurally (without OOP/classes) in a single sequence using the
**Grid layout**.
### Prerequisites
Before running the script, make sure you have the MySQL connector installed:
pip install mysql-connector-python
### Complete Python Code
'''

import csv
import tkinter as tk
from tkinter import messagebox, ttk
import mysql.connector

# =====
# DATABASE INITIALIZATION
# =====

def get_db_connection():
    # Update host, user, and password according to your MySQL setup
    return mysql.connector.connect(
        host="localhost", user="root", password="bismillah", database="Exam"
    )

def init_db():
    try:
        conn = mysql.connector.connect(
            host="localhost", user="root", password="bismillah"
        )
        cursor = conn.cursor()
        cursor.execute("CREATE DATABASE IF NOT EXISTS Exam")
        cursor.execute("USE Exam")
        cursor.execute(
            """
            CREATE TABLE IF NOT EXISTS students (
                roll_no INT PRIMARY KEY,
                name VARCHAR(100),
                gender VARCHAR(10),
                age INT,
                city VARCHAR(50),
                class VARCHAR(20),
                sub1 INT,
                sub2 INT,
                sub3 INT,
                sub4 INT,
                sub5 INT,
                sub6 INT
            )
            """
        )
        conn.commit()
        conn.close()
    
```

```

except mysql.connector.Error as err:
    messagebox.showerror(
        "Database Error", f"Failed to initialize database: {err}"
    )

init_db()

# =====
# MAIN APPLICATION WINDOW (Screen 1)
# =====

root = tk.Tk()
root.title("Exam Management System")
# root.geometry("450x350")

# =====
# FORM 1: ENTER STUDENT DATA
# =====

def open_form1():
    f1 = tk.Toplevel(root)
    f1.title("Form 1 - Enter Student Data")

    # Labels and Entries
    tk.Label(f1, text="Roll No:").grid(
        row=0, column=0, padx=5, pady=5, sticky="e"
    )
    e_roll = tk.Entry(f1)
    e_roll.grid(row=0, column=1, padx=5, pady=5)

    tk.Label(f1, text="Name:").grid(row=1, column=0, padx=5, pady=5, sticky="e")
    e_name = tk.Entry(f1)
    e_name.grid(row=1, column=1, padx=5, pady=5)

    tk.Label(f1, text="Gender:").grid(
        row=2, column=0, padx=5, pady=5, sticky="e"
    )
    e_gender = tk.Entry(f1)
    e_gender.grid(row=2, column=1, padx=5, pady=5)

    tk.Label(f1, text="Age:").grid(row=3, column=0, padx=5, pady=5, sticky="e")
    e_age = tk.Entry(f1)
    e_age.grid(row=3, column=1, padx=5, pady=5)

    tk.Label(f1, text="City:").grid(row=4, column=0, padx=5, pady=5, sticky="e")
    e_city = tk.Entry(f1)
    e_city.grid(row=4, column=1, padx=5, pady=5)

    tk.Label(f1, text="Class:").grid(
        row=5, column=0, padx=5, pady=5, sticky="e"
    )
    e_class = tk.Entry(f1)

```

```

e_class.grid(row=5, column=1, padx=5, pady=5)

tk.Label(f1, text="Subject 1:").grid(
    row=6, column=0, padx=5, pady=5, sticky="e"
)
e_s1 = tk.Entry(f1)
e_s1.grid(row=6, column=1, padx=5, pady=5)

tk.Label(f1, text="Subject 2:").grid(
    row=7, column=0, padx=5, pady=5, sticky="e"
)
e_s2 = tk.Entry(f1)
e_s2.grid(row=7, column=1, padx=5, pady=5)

tk.Label(f1, text="Subject 3:").grid(
    row=8, column=0, padx=5, pady=5, sticky="e"
)
e_s3 = tk.Entry(f1)
e_s3.grid(row=8, column=1, padx=5, pady=5)

tk.Label(f1, text="Subject 4:").grid(
    row=9, column=0, padx=5, pady=5, sticky="e"
)
e_s4 = tk.Entry(f1)
e_s4.grid(row=9, column=1, padx=5, pady=5)

tk.Label(f1, text="Subject 5:").grid(
    row=10, column=0, padx=5, pady=5, sticky="e"
)
e_s5 = tk.Entry(f1)
e_s5.grid(row=10, column=1, padx=5, pady=5)

tk.Label(f1, text="Subject 6:").grid(
    row=11, column=0, padx=5, pady=5, sticky="e"
)
e_s6 = tk.Entry(f1)
e_s6.grid(row=11, column=1, padx=5, pady=5)

def save_data():
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        query = """INSERT INTO students
                    (roll_no, name, gender, age, city, class, sub1, sub2, sub3, sub4,
sub5, sub6)
                    VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s, %s)"""
        vals = (
            int(e_roll.get()),
            e_name.get(),
            e_gender.get(),
            int(e_age.get()),
            e_city.get(),
            e_class.get(),
            int(e_s1.get()),

```

```

        int(e_s2.get()),
        int(e_s3.get()),
        int(e_s4.get()),
        int(e_s5.get()),
        int(e_s6.get()),
    )
    cursor.execute(query, vals)
    conn.commit()
    conn.close()
    messagebox.showinfo("Success", "Student Saved Successfully!")
    f1.destroy()
except Exception as e:
    messagebox.showerror("Error", f"Failed to save data: {e}")

tk.Button(f1, text="Save Student", command=save_data).grid(
    row=12, column=0, columnspan=2, pady=10
)

# =====
# FORM 2: DISPLAY INDIVIDUAL STUDENT DATA
# =====

def open_form2():
    f2 = tk.Toplevel(root)
    f2.title("Form 2 - Display Student Data")

    tk.Label(f2, text="Enter Roll No:").grid(
        row=0, column=0, padx=5, pady=5, sticky="e"
    )
    e_search_roll = tk.Entry(f2)
    e_search_roll.grid(row=0, column=1, padx=5, pady=5)

    # Frame to show details
    display_frame = tk.Frame(f2)
    display_frame.grid(row=2, column=0, columnspan=2, pady=10)

    def search_student():
        # Clear previous records in display frame
        for widget in display_frame.winfo_children():
            widget.destroy()

        try:
            conn = get_db_connection()
            cursor = conn.cursor()
            cursor.execute(
                "SELECT * FROM students WHERE roll_no = %s",
                (e_search_roll.get(),),
            )
            row = cursor.fetchone()
            conn.close()

            if row:

```

```

        labels = [
            "Roll No",
            "Name",
            "Gender",
            "Age",
            "City",
            "Class",
            "Sub 1",
            "Sub 2",
            "Sub 3",
            "Sub 4",
            "Sub 5",
            "Sub 6",
        ]
        for i in range(len(labels)):
            tk.Label(
                display_frame,
                text=f"{labels[i]}:",
                font=("Arial", 9, "bold"),
            ).grid(row=i, column=0, sticky="e", padx=5, pady=2)
            tk.Label(display_frame, text=str(row[i])).grid(
                row=i, column=1, sticky="w", padx=5, pady=2
            )
        else:
            messagebox.showwarning(
                "Not Found", "No record found for this Roll Number."
            )
    except Exception as e:
        messagebox.showerror("Error", f"Search failed: {e}")

tk.Button(f2, text="Search", command=search_student).grid(
    row=1, column=0, columnspan=2, pady=5
)

# =====
# FORM 3: UPDATE STUDENT DATA
# =====

def open_form3():
    f3 = tk.Toplevel(root)
    f3.title("Form 3 - Update Student Data")

    tk.Label(f3, text="Enter Roll No:").grid(
        row=0, column=0, padx=5, pady=5, sticky="e"
    )
    e_search = tk.Entry(f3)
    e_search.grid(row=0, column=1, padx=5, pady=5)

    # Placeholders for entry widgets
    entries = {}

    def fetch_and_populate():

```

```

try:
    conn = get_db_connection()
    cursor = conn.cursor()
    cursor.execute(
        "SELECT * FROM students WHERE roll_no = %s", (e_search.get(),)
    )
    row = cursor.fetchone()
    conn.close()

    if row:
        fields = [
            "Roll No",
            "Name",
            "Gender",
            "Age",
            "City",
            "Class",
            "Sub 1",
            "Sub 2",
            "Sub 3",
            "Sub 4",
            "Sub 5",
            "Sub 6",
        ]
        for i, field in enumerate(fields):
            tk.Label(f3, text=f"{field}:").grid(
                row=i + 2, column=0, padx=5, pady=2, sticky="e"
            )
            ent = tk.Entry(f3)
            ent.grid(row=i + 2, column=1, padx=5, pady=2)
            ent.insert(0, str(row[i]))
            if i == 0: # Roll No Primary Key should remain disabled
                ent.config(state="disabled")
            entries[field] = ent

            tk.Button(f3, text="Update Data", command=update_data).grid(
                row=15, column=0, columnspan=2, pady=10
            )
        else:
            messagebox.showwarning("Not Found", "Roll Number not found!")
    except Exception as e:
        messagebox.showerror("Error", f"Failed to fetch record: {e}")

def update_data():
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        query = """UPDATE students SET
            name=%s, gender=%s, age=%s, city=%s, class=%s,
            sub1=%s, sub2=%s, sub3=%s, sub4=%s, sub5=%s, sub6=%s
            WHERE roll_no=%s"""
        vals = (
            entries["Name"].get(),
            entries["Gender"].get(),

```

```

        int(entries["Age"].get()),
        entries["City"].get(),
        entries["Class"].get(),
        int(entries["Sub 1"].get()),
        int(entries["Sub 2"].get()),
        int(entries["Sub 3"].get()),
        int(entries["Sub 4"].get()),
        int(entries["Sub 5"].get()),
        int(entries["Sub 6"].get()),
        int(entries["Roll No"].get()),
    )
    cursor.execute(query, vals)
    conn.commit()
    conn.close()
    messagebox.showinfo("Success", "Record updated successfully!")
    f3.destroy()
except Exception as e:
    messagebox.showerror("Error", f"Failed to update record: {e}")

tk.Button(f3, text="Fetch Record", command=fetch_and_populate).grid(
    row=1, column=0, columnspan=2, pady=5
)

# =====
# FORM 4: DELETE STUDENT DATA
# =====

def open_form4():
    f4 = tk.Toplevel(root)
    f4.title("Form 4 - Delete Student Data")

    tk.Label(f4, text="Enter Roll No to Delete:").grid(
        row=0, column=0, padx=5, pady=10, sticky="e"
    )
    e_del_roll = tk.Entry(f4)
    e_del_roll.grid(row=0, column=1, padx=5, pady=10)

    def delete_data():
        roll = e_del_roll.get()
        if not roll:
            messagebox.showwarning("Warning", "Please enter a Roll Number!")
            return

        confirm = messagebox.askyesno(
            "Confirm", f"Are you sure you want to delete Roll No: {roll}?"
        )
        if confirm:
            try:
                conn = get_db_connection()
                cursor = conn.cursor()
                cursor.execute(
                    "DELETE FROM students WHERE roll_no = %s", (roll,)
                )

```

```

        )
        conn.commit()

        if cursor.rowcount > 0:
            messagebox.showinfo(
                "Success", "Student record deleted successfully!"
            )
            f4.destroy()
        else:
            messagebox.showwarning(
                "Not Found", "Roll Number does not exist!"
            )

        conn.close()
    except Exception as e:
        messagebox.showerror("Error", f"Deletion failed: {e}")

tk.Button(f4, text="Delete Record", command=delete_data).grid(
    row=1, column=0, columnspan=2, pady=10
)

# =====
# FORM 5: DISPLAY ALL DATA & EXPORT TO CSV
# =====

def open_form5():
    f5 = tk.Toplevel(root)
    f5.title("Form 5 - All Students & Export CSV")
    f5.geometry("900x400")

    # Scrollable Table Frame setup
    table_frame = tk.Frame(f5)
    table_frame.grid(row=0, column=0, padx=10, pady=10, sticky="nsew")

    scroll_x = ttk.Scrollbar(table_frame, orient="horizontal")
    scroll_y = ttk.Scrollbar(table_frame, orient="vertical")

    cols = (
        "Roll No",
        "Name",
        "Gender",
        "Age",
        "City",
        "Class",
        "Sub 1",
        "Sub 2",
        "Sub 3",
        "Sub 4",
        "Sub 5",
        "Sub 6",
    )

    tree = ttk.Treeview(

```



```

        table_frame,
        columns=cols,
        show="headings",
        xscrollcommand=scroll_x.set,
        yscrollcommand=scroll_y.set,
    )

    scroll_x.pack(side="bottom", fill="x")
    scroll_y.pack(side="right", fill="y")
    scroll_x.config(command=tree.xview)
    scroll_y.config(command=tree.yview)

    for col in cols:
        tree.heading(col, text=col)
        tree.column(col, width=70, anchor="center")

    tree.pack(fill="both", expand=True)

    # Fetch and load data into Treeview
    try:
        conn = get_db_connection()
        cursor = conn.cursor()
        cursor.execute("SELECT * FROM students")
        rows = cursor.fetchall()
        for row in rows:
            tree.insert("", "end", values=row)
        conn.close()
    except Exception as e:
        messagebox.showerror("Error", f"Failed to fetch data: {e}")

    # Export to CSV logic
    def export_to_csv():
        try:
            conn = get_db_connection()
            cursor = conn.cursor()
            cursor.execute("SELECT * FROM students")
            rows = cursor.fetchall()

            with open(
                "student_data.csv", "w", newline="", encoding="utf-8"
            ) as file:
                writer = csv.writer(file)
                writer.writerow(cols) # Write header row
                writer.writerows(rows) # Write data rows

            conn.close()
            messagebox.showinfo(
                "Success", "Data exported successfully to 'student_data.csv'!"
            )
        except Exception as e:
            messagebox.showerror("Error", f"Export failed: {e}")

    tk.Button(f5, text="Export All Data to CSV", command=export_to_csv).grid(

```

```

        row=1, column=0, pady=10
    )

# =====
# SCREEN 1 LAYOUT (MAIN MENU)
# =====

tk.Label(
    root, text="Exam Management System", font=("Arial", 16, "bold")
).grid(row=0, column=0, columnspan=2, pady=20, padx=50)

tk.Button(
    root, text="1. Enter Student Data", width=30, command=open_form1
).grid(row=1, column=0, pady=5)
tk.Button(
    root, text="2. Display Student Data", width=30, command=open_form2
).grid(row=2, column=0, pady=5)
tk.Button(
    root, text="3. Update Student Data", width=30, command=open_form3
).grid(row=3, column=0, pady=5)
tk.Button(
    root, text="4. Delete Student Data", width=30, command=open_form4
).grid(row=4, column=0, pady=5)
tk.Button(
    root, text="5. Display All & Export to CSV", width=30, command=open_form5
).grid(row=5, column=0, pady=5)

root.mainloop()

```

Exam Management System

Exam Management System

1. Enter Student Data

2. Display Student Data

3. Update Student Data

4. Delete Student Data

5. Display All & Export to CSV

For...

Roll No:

Name:

Gender:

Age:

City:

Class:

Subject 1:

Subject 2:

Subject 3:

Subject 4:

Subject 5:

Subject 6:

Save Student

Form 2 - Display Student Data

Enter Roll No:

Search

Form 3 - Update Student Data

Enter Roll No:

Fetch Record

 Form 4 - Delete Student Data



Enter Roll No to Delete:

Delete Record

 Form 5 - All Students & Export CSV



Roll No	Name	Gender	Age	City	Class	Sub 1	Sub 2	Sub 3	Sub 4	Sub 5	Sub 6

Export All Data to CSV